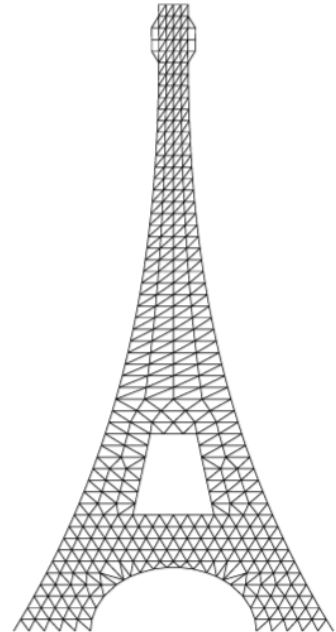


3. Stora matriser

I många tillämpningar måste man lösa *stora* linjära ekvationssystem, med miljontals obekanta. Det är i dessa fall som effektiva algoritmer blir viktiga att använda. Som exempel ska ni här räkna på ett komplicerat fackverk: en modell av Eiffeltornet. Ett fackverk består av stänger förenade genom leder i ett antal noder. Ni ska beräkna deformationen av fackverket när noderna belastas av yttre krafter. Ekvationerna för deformationen härleds i hållfasthetsläran, och baseras på att förskjutningarna i varje nod är små, och att Hookes lag gäller för förlängningen av varje stång. Ett dokument med härledningar finns i Canvas.

I slutändan får man ett linjärt ekvationssystem på formen $A\mathbf{x} = \mathbf{b}$. När antalet noder i fackverket är n kommer antalet obekanta vara $2n$ och $A \in \mathbb{R}^{2n \times 2n}$. Matrisen A brukar kallas *styvhetsmatrisen*.



Högerledet $\mathbf{b}$ innehåller de givna yttre krafterna som verkar på noderna,

$$\mathbf{b} = (F_1^x, F_1^y, F_2^x, F_2^y, \dots, F_n^x, F_n^y)^T, \quad \mathbf{b} \in \mathbb{R}^{2n},$$

där $\mathbf{F}_j = (F_j^x, F_j^y)^T$ är kraften i nod j . Lösningen $\mathbf{x}$ innehåller de resulterande (obekanta) förskjutningarna,

$$\mathbf{x} = (\Delta x_1, \Delta y_1, \Delta x_2, \Delta y_2, \dots, \Delta x_n, \Delta y_n)^T, \quad \mathbf{x} \in \mathbb{R}^{2n}.$$

Här är alltså $(\Delta x_j, \Delta y_j)^T$ förskjutningen av nod j när fackverket belastas med krafterna i $\mathbf{b}$.

I Canvas finns filerna `eiffel1.mat`, `eiffel2.mat`, ..., `eiffel6.mat`. De innehåller sex olika modeller av Eiffeltornet med växande detaljrikedom ($n = 261, 399, 561, 1592, 2290, 4330$). Varje modell består av nodkoordinater i vektorerna `xnod`, `ynod`, stångindex i matrisen `bars` (används bara för plotningen) och styvhetsmatrisen `A`.

LINJÄRA EKVATIONSSYSTEM

a) Förberedande arbete

Ladda in den minsta modellen `eiffel1.mat` i MATLAB med kommandot `load`. Hämta funktionsfilen `trussplot.m` från Canvas och anropa den med `trussplot(xnod,ynod,bars)` för att plotta tornet. Välj en av noderna och belast den med en kraft snett nedåt höger med beloppet $\sqrt{2}$. (Sätt $F_j^x = 1$ och $F_j^y = -1$ för något j . Låt resten av elementen i $\mathbf{b}$ vara noll, dvs `b=zeros(2*n,1)`; `b(j*2-1)=1`; `b(j*2)=-1`; i MATLAB.) Lös systemet $A\mathbf{x} = \mathbf{b}$ med backslash för att få fram förskjutningarna i alla punkter. Beräkna de nya koordinaterna för det belastade tornet, $x_j^{\text{bel}} = x_j + \Delta x_j$, etc.:

```
xbel = xnod + x(1:2:end); ybel = ynod + x(2:2:end);
```

Plotta det belastade tornet. Använd `hold on` för att plotta de två tornen ovanpå varandra i samma figur. Markera vilken nod ni valt med en asterisk `*` i figuren.

TIDSKOMPLEXITET

- b) Backslash-kommandot i MATLAB använder normalt vanlig gausseliminering. Undersök hur tidsåtgången för gausseliminering beror på systemmatrisens storlek genom att lösa ekvationssystemet $A\mathbf{x} = \mathbf{b}$ med ett godtyckligt valt högerled $\mathbf{b}$ (tex med `b=randn(,1)`) för var och en av de sex modellerna. Använd MATLAB-kommandona `tic` och `toc` (`help tic` ger mer info).^a Plotta tidsåtgången mot antal obekanta $N = 2n$ i en `loglog`-plot; använd också `grid on`. Hur ska tidsåtgången bero på enligt teorin? Stämmer det överens med din plot?

LU-FAKTORISERING OCH GLESA MATRISER

- c) Ni ska nu räkna ut vilken nod i fackverket som är mest känslig för belastning snett nedåt höger. Tidsåtgången för ett par olika metoder ska jämföras.

För att hitta den sökta noden måste man lösa ekvationssystemet en gång för varje nod. Mer precist behöver man bestämma $\mathbf{x}_j$ så att

$$A\mathbf{x}_j = \mathbf{b}_j, \quad j = 1, \dots, n,$$

där $\mathbf{b}_j$ betecknar högerledsvektorn som motsvarar belastning snett nedåt höger, dvs som i deluppgift (a), $F_j^x = 1$, $F_j^y = -1$ och all övriga element noll. Man definierar sedan den totala förskjutningen T_j som den euklidiska normen av $\mathbf{x}_j$,

$$T_j := \|\mathbf{x}_j\|, \quad \|\mathbf{x}\| = \left(\sum_{k=1}^n \Delta x_k^2 + \Delta y_k^2 \right)^{1/2}.$$

Den känsligaste noden är den som har störst T_j .

- (i) Skriv ett MATLAB-program som loopar igenom alla noderna i fackverket, beräknar $\mathbf{x}_j$ och T_j samt hittar den känsligaste noden. Använd backslash här för att lösa ekvationssystemen och kommandot `norm` för att beräkna T_j . Förvissa dig om rimligheten i resultatet genom att tex plotta tornet och markera den känsligaste noden i plotten. (Testkör med minsta modellen `eiffel1.mat`.)
- (ii) Lägg till kod som mäter tiden för beräkningarna. Använd `tic` och `toc` som tidigare. Inkludera `inte` plottning och inladdning av filer i tidsmätningen.
- (iii) Den naiva metoden att lösa ekvationssystemen med backslash varje gång blir mycket tidskrävande för de större modellerna. Använd istället LU-faktorisering (MATLAB-kommandot `lu`) för att snabba upp beräkningarna. LU-faktoriseringen ska bara göras en gång, utanför loopen, men den *ska inkluderas* i tidsmätningen. Verifiera att metoden ger samma resultat som den naiva metoden och är snabbare.
- (iv) När en matris är *gles* kan man lösa ekvationssystemet med effektivare metoder än standard gausseliminering. (Använd kommandot `spy(A)` för att förvissa dig om att A i det här fallet faktiskt är gles (bandad).) Konvertera A till glest format med kommandot `A=sparse(A)`. (Inkluderas `inte` i tidsmätningen!) MATLAB använder då bättre metoder när backslash och `lu` anropas. Verifiera att beräkningarna går snabbare och att du fortfarande får samma resultat.

Gå igenom de sex modellerna `eiffel1.mat`, ..., `eiffel6.mat` och mät hur lång tid det tar att lösa problemet med de fyra olika metoderna som diskuterats ovan.^b Sammanfatta resultaten i en tabell enligt nedan. Tabellen ska skickas in.

	Naiv	LU	Gles (ej LU)	Gles+LU
<code>eiffel1.mat</code>				
<code>eiffel2.mat</code>				
⋮				
<code>eiffel6.mat</code>				

- Varför går det snabbare att lösa problemet med LU-faktorisering?
- Vilken metod löser problemet snabbast? (Med/utan LU? Full/gles lösare?)
- För vilken modell blir tidsvinsten störst? Varför?

^aFör att få bra noggrannhet i tidsmätningen (speciellt om den är kort) kan man mäta totaltiden för flera upprepade beräkningar, och sedan dividera tiden med antalet upprepningar.

^bNi kan hoppa över de fall som tar mer än en minut.